◆ DAY 5

Git 团队协作与全栈应用部署

从 Git 小团队协作到宝塔、Docker、域名与 HTTPS

主讲：Tauh

DAY 1
认知与环境

DAY 2
前端 + Skill 机制

DAY 3
后端开发与模型接入

DAY 4
从 LLM 到 Agent - RAG

DAY 5
Git 协作与全栈部署

课程地图

Git 协作	分支、Commit、Push、冲突处理与主分支治理
宝塔部署	服务器、Nginx、前端构建、FastAPI / Node 进程守护
Docker	Dockerfile、Compose；结束页后附 Qwen3.6 本地模型部署
公网运维	DNS、域名、反向代理、SSL、日志与故障定位

01

开场与课程主线

Git 协作、服务器部署、域名与 HTTPS

1.1	今天只讲两条主线	04
1.2	演示项目与部署结构	05

1.1 今天只讲两条主线

Git 团队协作 + 全栈应用部署

主线 A · Git 协作

- 用分支隔离任务
- 用 Commit 记录可解释的变化
- 用 Push 共享分支，同伴检查后合并
- 用规则保护主分支

主线 B · 公网部署

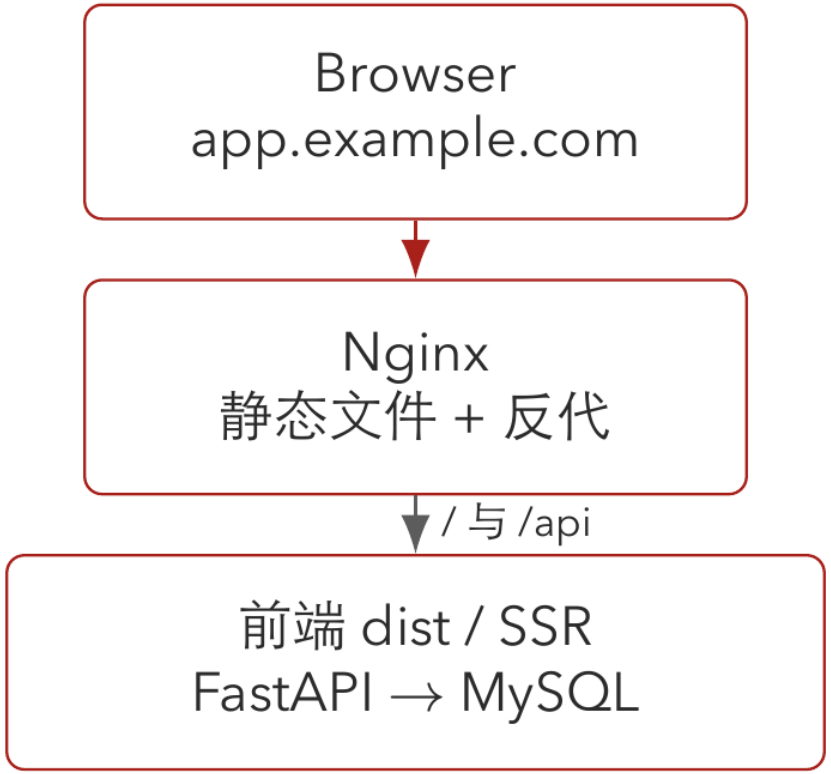
- 服务器持续运行进程
- Nginx 接住公网请求
- 宝塔或 Docker 管理应用
- 域名 + SSL 提供可信入口

贯穿项目 Node 技术栈前端 + Python FastAPI 后端 + MySQL，可替换为 Node 后端

1.2 演示项目与部署结构

Node 前端 + FastAPI 后端 + MySQL

- **前端**：Vite / React / Vue 等 Node 技术栈
- **包管理**：pnpm + `pnpm-lock.yaml`
- **后端**：FastAPI + uv + `uv.lock`
- **数据**：MySQL，生产环境不暴露公网端口
- **入口**：/ 访问前端，/api 访问后端



02

Git 核心模型与本地工作流

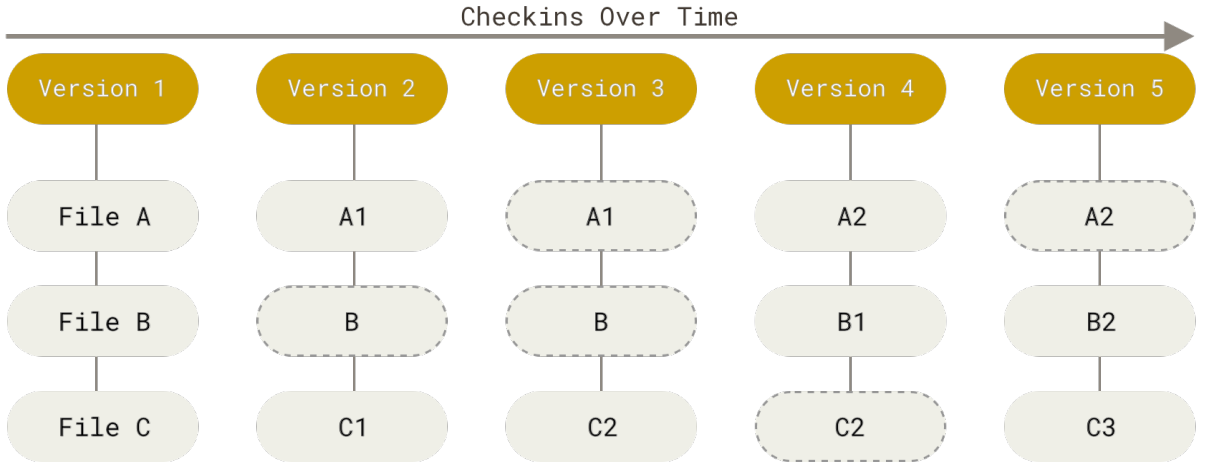
先理解 Snapshot、Commit 与 Branch，再进入命令操作

2.1	Git 保存的是一串项目快照	07
2.2	Commit 是一段可追踪的历史	08
2.3	Git 的三个区域	09
2.4	分支是什么	10
2.5	第一次使用 Git 的身份配置	11
2.6	获取仓库：init 与 clone	12
2.7	一次最小提交循环	13
2.8	Commit 写入本地仓库	14
2.9	Commit 和 Push 是两步	15
2.10	用 log 与 show 阅读历史	16
2.11	分支的日常命令	17
2.12	撤销操作先判断“是否已经共享”	18

2.1 Git 保存的是一串项目快照

先理解 Git 记录什么，再学习具体命令

- 每次 Commit 都指向当时的完整项目快照。
- 没有变化的文件会复用已有对象，不会重复保存。
- Commit 串联起来，就形成可追踪、可比较的项目历史。



Git 以快照组织历史，而不是保存一堆项目副本。

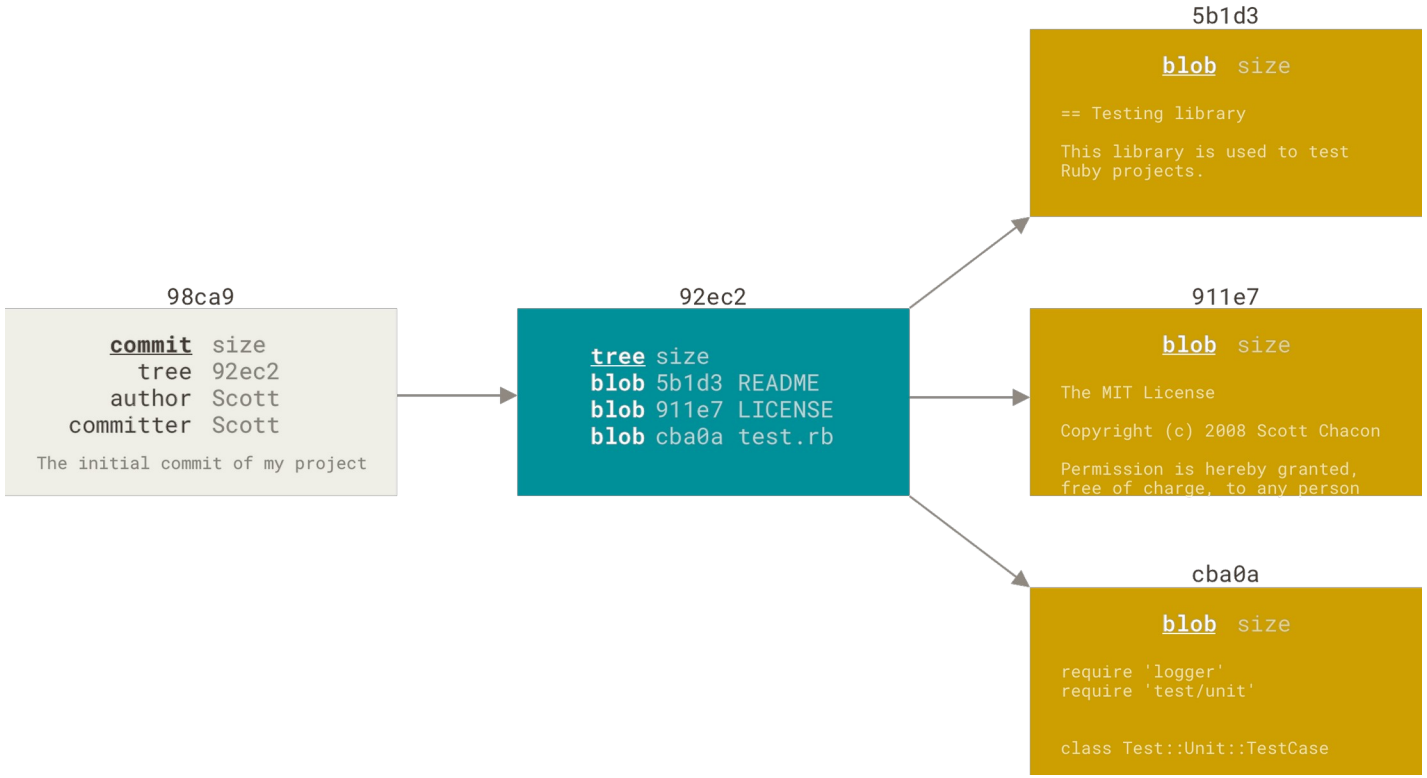
来源：Pro Git · Snapshots, Not Differences

核心价值 随时回答“谁在什么时候，为什么改了什么”

2.2 Commit 是一段可追踪的历史

快照 + 作者 + 时间 + 说明 + 父 Commit

- Commit 用唯一 ID 标识，例如 `0fd229a`。
- 父 Commit 把一次次快照连接成历史链。
- 好 Commit 应当**小、完整、能读懂**。



Commit 指向 Tree，Tree 再描述目录和文件对象。

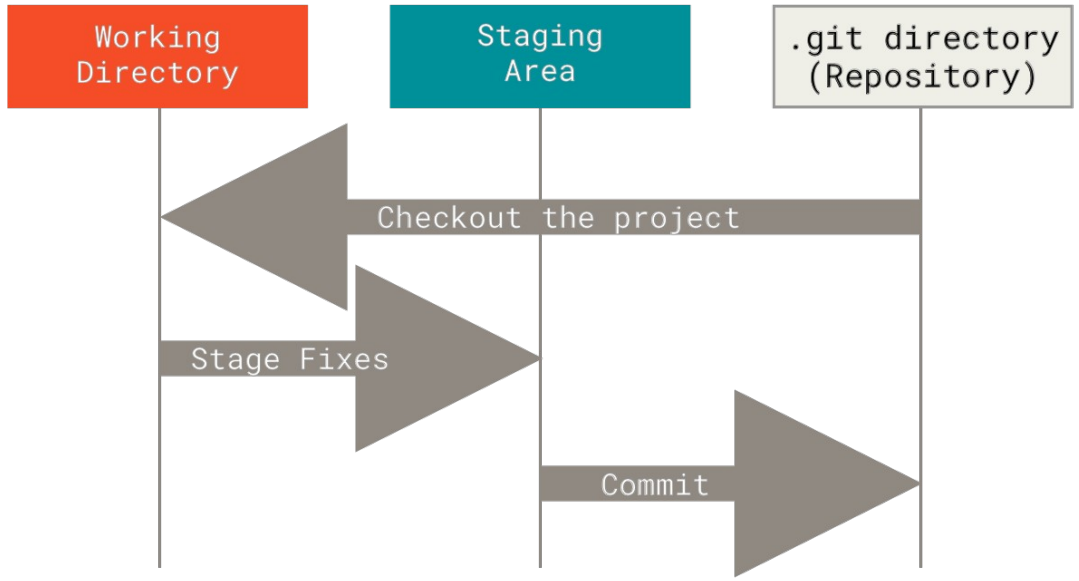
来源：Pro Git · Git Objects

示例

```
fix(api): handle 401 response
```

2.3 Git 的三个区域

文件先变化，再被选择，最后写入历史



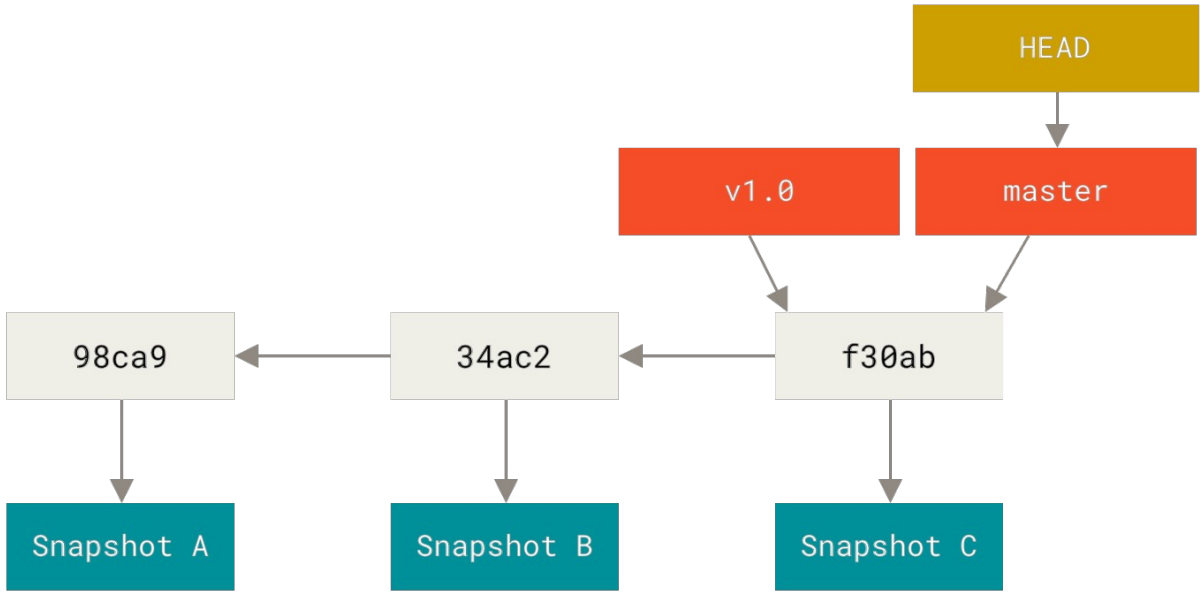
来源：Pro Git · The Three States

- **工作区**：正在编辑的文件。
- **暂存区**：准备进入下一次 Commit 的变化。
- **本地仓库**：已经提交的历史。

路径 `git add` 选择；`git commit` 写入

2.4 分支是什么

指向某个 Commit 的可移动标签



来源：Pro Git · Branches in a Nutshell

- 分支不是项目副本，而是指向 Commit 的轻量指针。
- 新 Commit 会推动当前分支向前移动。
- 每个任务使用独立分支，方便并行开发与安全放弃。

2.5 第一次使用 Git 的身份配置

作者信息会写入每个 Commit

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

git config --global init.defaultBranch main
git config --global --list
```

- 公司与个人账号需要区分时，可在项目目录使用不带 `--global` 的配置。
- 提交邮箱应与 GitHub 账号关联，才能正确显示贡献身份。
- 认证使用 SSH Key 或受控 Token；密码与 Token 留在受控环境。

2.6 获取仓库：init 与 clone

一个创建新历史，一个复制已有历史

新项目

```
mkdir customer-service
cd customer-service
git init
git status
```

已有项目

```
git clone git@github.com:org/app.git
cd app
git remote -v
```

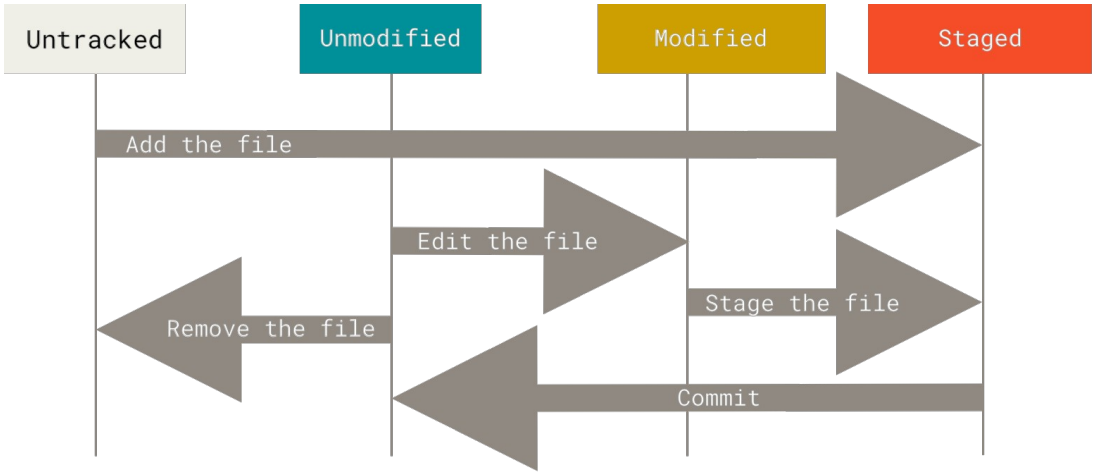
团队默认 使用 Clone 获取完整历史与远程仓库配置

2.7 一次最小提交循环

修改文件、选择变化、检查、Commit

```
git status
git add docs/git-demo-notes.md
git diff --cached
git commit -m "docs: add Git demo notes"
```

- 先选择，再检查，最后提交。
- Commit 只写入本机仓库。



文件会在未跟踪、已修改、已暂存之间流转。

来源：Pro Git · Recording Changes

2.8 Commit 写入本地仓库

先看输出，确认真正记录了什么

```
$ git diff --cached --stat
 docs/git-demo-notes.md | 13 ++++++++++++
 1 file changed, 13 insertions(+)

$ git commit -m "docs: add Git workflow demo notes"
[main 0fd229a] docs: add Git workflow demo notes
 1 file changed, 13 insertions(+)
 create mode 100644 docs/git-demo-notes.md
```

此刻 Commit 已存在于本地，GitHub 仍停留在上一个 Commit

2.9 Commit 和 Push 是两步

Commit 记录本地历史，Push 才把历史同步到 GitHub

Push 前

```
$ git status -sb
## main...origin/main [ahead 1]

$ git log --oneline --decorate -2
0fd229a (HEAD -> main) docs: add
  Git workflow demo notes
8ce893a (origin/main) feat: add
  deployment and operations lessons
```

Push 后

```
$ git push
 8ce893a..0fd229a  main -> main

$ git status -sb
## main...origin/main
```

2.10 用 log 与 show 阅读历史

先看提交顺序，再查看某次变化

```
$ git log --oneline --decorate --graph -4
* 0fd229a (HEAD -> main, origin/main)
  docs: add Git workflow demo notes
* 8ce893a feat: add deployment and operations lessons
* bef37be feat: add Git collaboration lessons
* ab2afa2 chore: initialize Day 5 beamer project

$ git show --stat 0fd229a
 docs/git-demo-notes.md | 13 ++++++++++++
 1 file changed, 13 insertions(+)
```

- `HEAD -> main, origin/main` 表示本地与 GitHub 已对齐。
- `git show 0fd229a` 查看某次提交的说明与完整 Diff。
- `git diff ab2afa2..0fd229a` 比较两个历史节点。

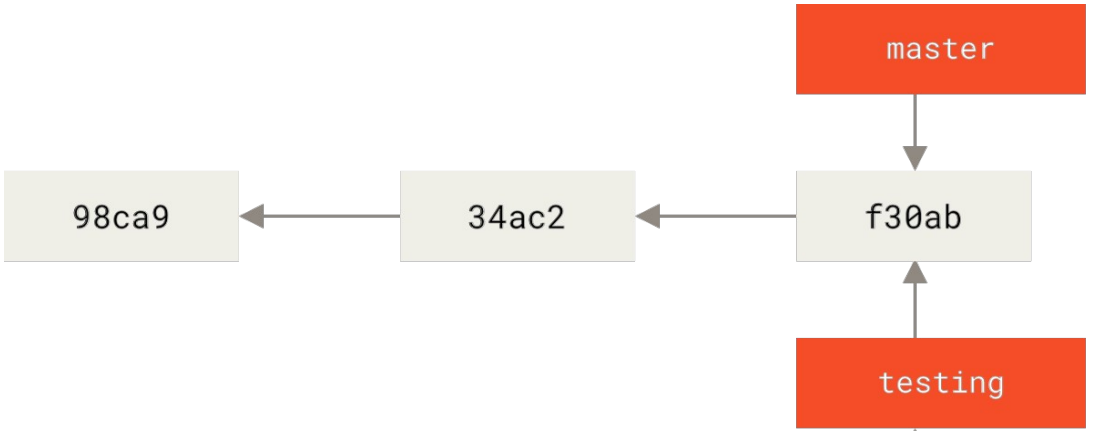
2.11 分支的日常命令

从最新 main 创建任务分支

```
git switch main
git pull --ff-only

git switch -c feature/login-api
# edit, test, commit...

git switch main
git branch -d feature/login-api
```



来源：Pro Git · Basic Branching

- 从最新 main 创建任务分支。
- 合并后及时删除过期分支。

2.12 撤销操作先判断“是否已经共享”

公开历史优先 revert，未共享历史才考虑重写

场景	推荐命令	影响
未暂存的文件改坏	`git restore file`	丢弃本地变化
误加入暂存区	`git restore --staged file`	保留文件变化
最近 Commit 信息错	`git commit --amend`	重写未共享提交
已推送的错误 Commit	`git revert <commit>`	新增反向提交，历史安全

红线 共享分支不执行 `reset --hard` 或普通 `push --force`

03

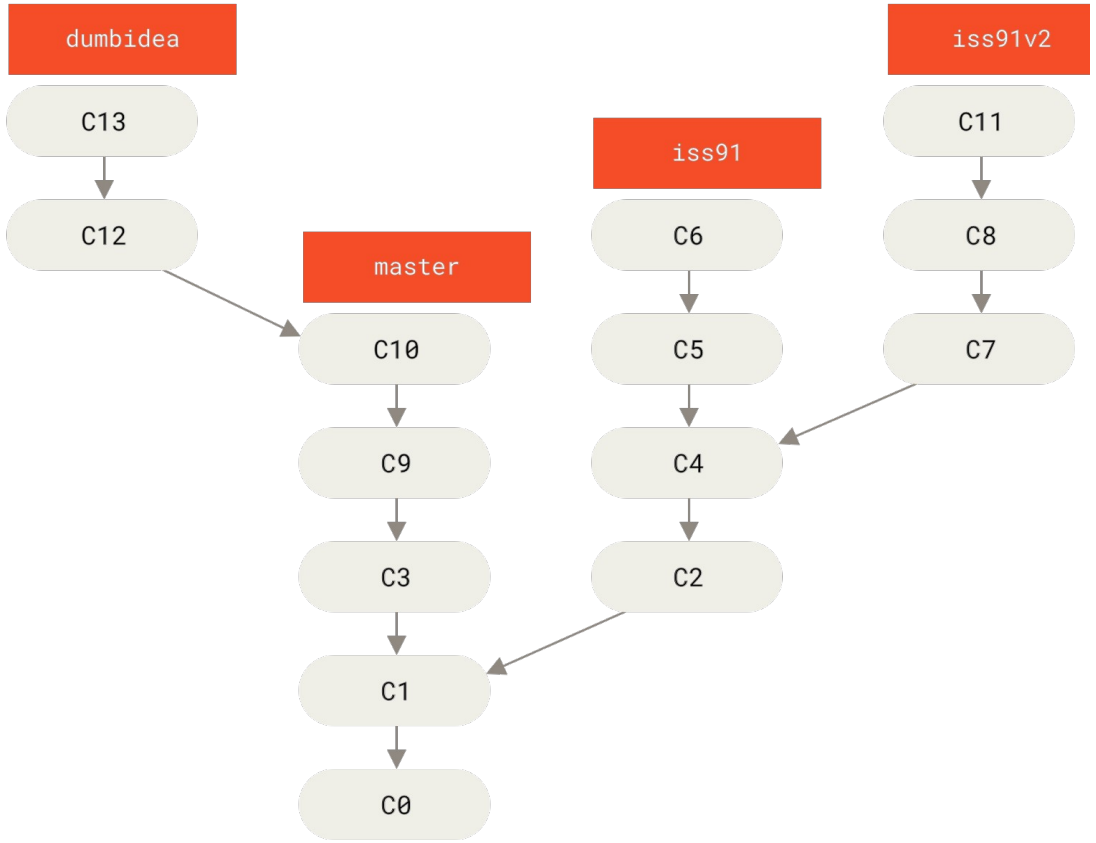
GitHub 小团队协作流程

任务分支、小步提交、同步 main、合并检查

3.1	小团队开发的完整流程	20
3.2	开发者的日常命令流	21
3.3	合并前快速检查	22
3.4	开发期间同步 main	23
3.5	密钥必须留在 Git 之外	24

3.1 小团队开发的完整流程

一个任务、一个分支、一个合并目标



来源：Pro Git · Topic Branches

1. 明确任务边界与负责人。
2. 从 main 创建短生命周期分支。
3. 小步 Commit，并 Push 共享。
4. 同伴快速检查后合并。
5. 删除已完成的功能分支。

3.2 开发者的日常命令流

同步 main、创建分支、提交、推送

```
git switch main
git pull --ff-only
git switch -c feature/chat-history

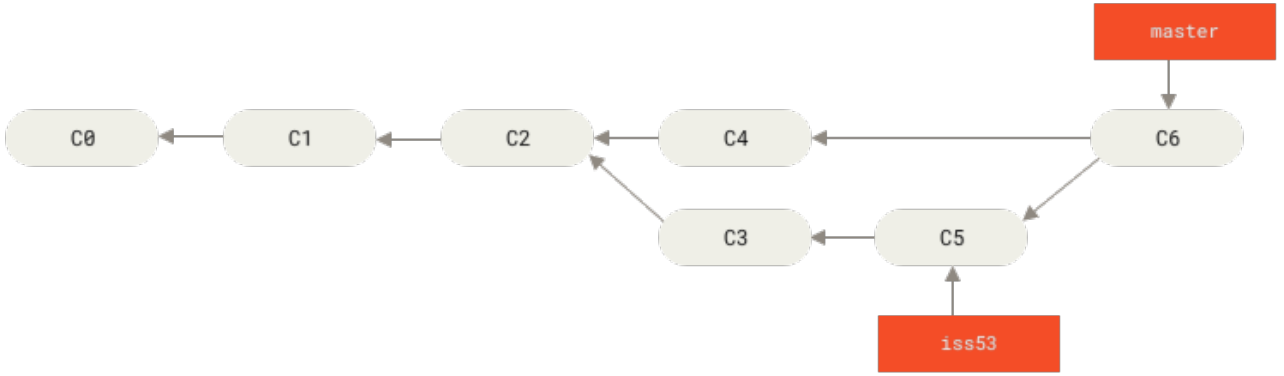
# edit and test
git add src/
git diff --cached
git commit -m "feat(chat): load conversation history"

git push -u origin feature/chat-history
```

下一步 在 GitHub 发起合并请求，由一名同伴快速检查后合并

3.3 合并前快速检查

围绕本次任务检查逻辑、测试、配置与密钥



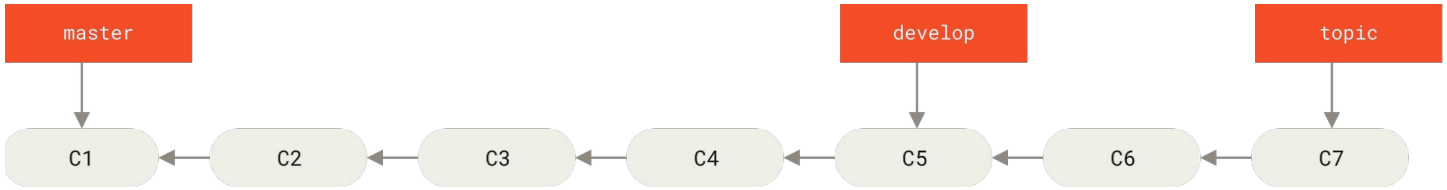
来源: Pro Git · Basic Merging

- 改动只包含当前任务。
- 核心逻辑与错误处理正确。
- 构建、测试或最小验证通过。
- 不包含 .env、密码和密钥。
- 接口、配置和迁移变化已说明。
- 合并后删除功能分支。

控制范围 一次合并只解决一个问题，降低检查与回滚成本

3.4 开发期间同步 main

小团队优先使用 Merge，规则保持统一



来源：Pro Git · Long-Running Branches

```
git fetch origin
git switch feature/chat-history
git merge origin/main

# resolve conflicts, test, commit
git push
```

- 功能分支开发超过半天时主动同步 main。
- 冲突在自己的分支解决并测试，避免把问题留到最终合并。

3.5 密钥必须留在 Git 之外

泄漏后立即轮换并清理历史

严禁提交

.env、数据库密码、JWT Secret、云厂商密钥、模型 API Key、私钥文件。

- 发现泄漏后立即吊销或轮换密钥，再处理仓库文件。
- 清理 Git 历史、Fork、缓存和 CI 日志，并检查泄漏原因。
- 部署时通过服务器环境变量、Secret 文件或平台 Secret 注入。

提交前检查 `git diff --cached` + Secret Scanner + 最小权限密钥

04

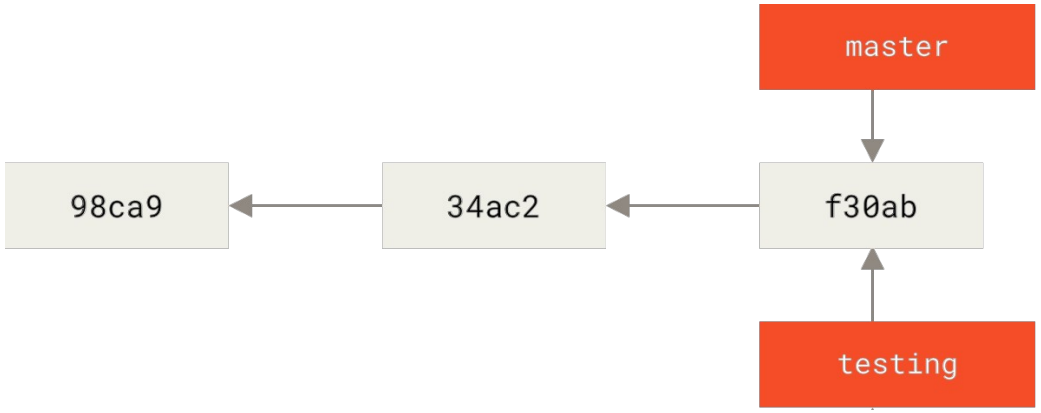
冲突处理、Review 与主分支治理

读标记、解冲突、保护 main

4.1	先学会读冲突标记	26
4.2	命令行解决冲突的标准步骤	27
4.3	GitHub 在线解决简单行冲突	28
4.4	如何减少冲突	29
4.5	Rebase 为什么需要谨慎	30
4.6	Rebase 后推送用 force-with-lease	31
4.7	主分支保护规则	32

4.1 先学会读冲突标记

当前分支、分隔线、待合并分支



来源：Pro Git · Divergent Branches

```
<<<<<<< HEAD
const apiBase = "/api";
=======
const apiBase = "http://localhost:8000";
>>>>>>> feature/api-config
```

- 先确认生产架构与配置约定，再选择或融合两侧代码。
- 删除全部标记后，必须重新运行测试与构建。

4.2 命令行解决冲突的标准步骤

定位、编辑、验证、继续

```
git status
# open conflicted files and resolve markers

pnpm lint
pnpm test
pnpm build

git add src/config/api.ts
git commit                  # merge workflow
# or: git rebase --continue
```

- `git status` 会列出所有未合并路径。
- 不确定时可用 `git merge --abort` 或 `git rebase --abort` 回到操作前。
- 解决冲突时必须理解两边业务意图。

4.3 GitHub 在线解决简单行冲突

复杂冲突回到本地测试



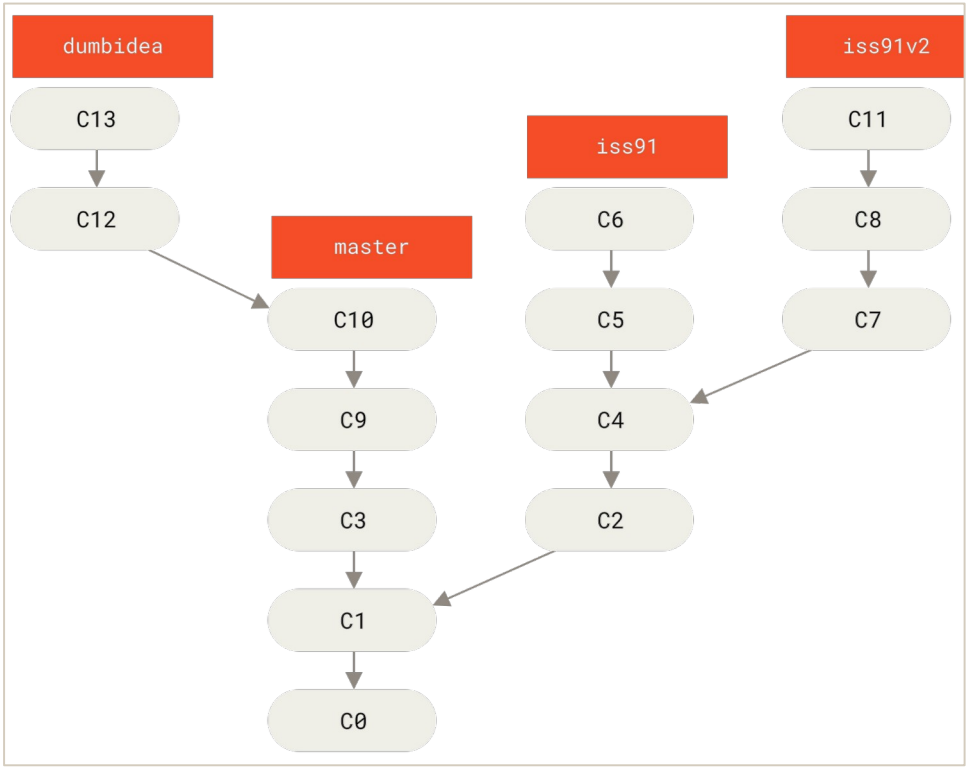
来源：GitHub Docs · Resolving a merge conflict

- 简单的竞争行可以使用网页冲突编辑器。
- 删除 / 重命名、生成文件、依赖锁文件等复杂冲突建议本地处理。
- 合并前必须验证构建、测试和关键业务流程。

4.4 如何减少冲突

重点是工作拆分、同步和接口约定

- 拆成小任务，使用短生命周期分支。
- 明确模块负责人，减少同时修改核心文件。
- 频繁同步 main，尽早暴露竞争变化。
- Schema、路由和公共配置先约定再编码。
- 格式化与依赖更新单独提交。

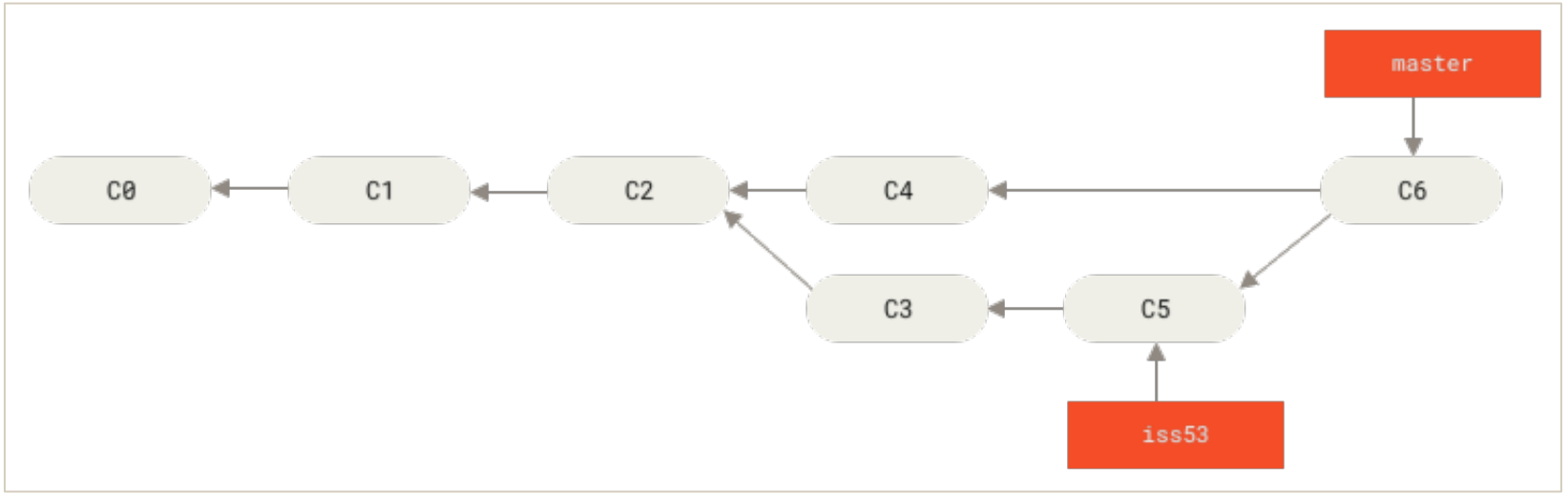


一个分支只承载一个清晰任务。

来源：Pro Git · Topic Branches

4.5 Rebase 为什么需要谨慎

它会把 Commit 复制到新的父节点上



来源：Pro Git · Basic Merging

- Merge 保留分叉历史；Rebase 会复制提交到新的父节点。
- Rebase 后 Commit ID 改变，远程推送可能需要历史重写。
- 只重写自己的功能分支；共享历史保持稳定。

4.6 Rebase 后推送用 force-with-lease

它会检查远程分支是否被别人更新

```
git fetch origin
git rebase origin/main

git push --force-with-lease
```

禁止使用 `git push --force` 可能覆盖同事刚推送的 Commit。

边界 如果团队尚未理解历史重写，统一用 Merge 同步 main 更安全

4.7 主分支保护规则

main 始终代表“可部署版本”

- 禁止直接 Push，所有变化通过受控合并流程进入 main。
- 至少一人 Approve，关键目录可配置 CODEOWNERS。
- 必须通过构建、测试、Lint 等状态检查。
- 必须解决 Review 对话后才能合并。
- 默认禁止 Force Push 与删除主分支。

来源：GitHub Docs · About protected branches

结果 规则把“团队口头约定”变成平台自动执行的门禁

05

服务器购买与部署架构

服务器配置、端口与部署路线

5.1	购买服务器	34
5.2	端口开放策略	35
5.3	宝塔与 Docker 的分工	36

5.1 购买服务器

选择地域、系统、配置与网络

地域	靠近用户；境内上线关注备案
系统	Ubuntu / Debian LTS 纯净镜像
配置	课堂建议 2C4G 起步
网络	公网 IPv4、带宽、22 / 80 / 443

来源：工信部·非经营性互联网信息服务备案管理办法

购买后记录

- 公网 IPv4、SSH 用户与密钥
- 实例名称与到期时间
- 云防火墙规则
- 控制台重置与救援入口

提醒：境内上线应依法判断备案或许可要求。

5.2 端口开放策略

公网保留 22、80、443

端口	典型用途	建议
22	SSH	仅管理使用；可限制来源 IP、使用密钥登录
80	HTTP	证书验证与跳转 HTTPS
443	HTTPS	用户正式访问入口
3000	Node / Next.js	默认仅本机或容器网络
8000	FastAPI	默认仅本机或容器网络
3306	MySQL	不向公网开放，应用走本机 / 内网

来源：阿里云轻量应用服务器·防火墙

5.3 宝塔与 Docker 的分工

可视化运维与可重复交付

维度	宝塔部署	Docker 部署
学习门槛	面板可视化，上手快	需要理解镜像、网络、卷
环境一致性	依赖服务器当前状态	镜像描述运行环境
迁移复现	需要记录手工步骤	Dockerfile + Compose 可重复
排错入口	面板日志 + 系统日志	docker logs + 容器状态
适用	新手、小型单机项目	团队交付、多服务、持续迭代

组合方式 宝塔管理 Nginx 与证书，Docker Compose 运行应用

06

宝塔部署的关键知识点

理解站点、进程、反向代理与配置

6.1	环境组件与职责	38
6.2	静态前端在宝塔里的本质	39
6.3	SPA 刷新 404 怎么修	40
6.4	FastAPI 在宝塔中的运行模型	41
6.5	进程守护需要配置什么	42
6.6	Nginx 把/api 转给 FastAPI	43
6.7	反向代理界面要看懂的字段	44
6.8	前后端联调的验证顺序	45
6.9	Node 后端与 FastAPI 的部署差异	46
6.10	Node 项目配置要认清的字段	47
6.11	环境变量如何放到服务器	48

6.1 环境组件与职责

先理解每一层负责什么

组件	用途	选择建议
Nginx	80 / 443、静态文件、反向代理	必装
Node.js	构建前端、运行 Node 后端 / SSR	安装项目要求的 LTS 版本
pnpm	Node 依赖与构建	通过 Corepack 或官方方式启用
Python + uv	FastAPI 运行环境	与 `pyproject.toml` 要求一致
MySQL	业务数据	本机部署时仅监听本地 / 内网

原则：项目版本来自仓库声明，生产升级要有计划。

6.2 静态前端在宝塔里的本质

Nginx 读取构建产物，不直接运行前端源码

- 网站根目录指向 `dist/` 等生产构建产物，而非 `src/` 或项目根目录。
- Vite / React / Vue 的静态产物由 Nginx 直接返回，不需要常驻 Node 进程。
- 前端请求使用相对路径 `/api`，方便由同域 Nginx 转发后端。
- 构建时环境变量会写入浏览器可见的静态文件，不能保存 Secret。
- 环境变量或源码变化后，需要重新构建并替换产物。

先判断 静态构建与 Next.js / Node SSR 的运行方式不同

6.3 SPA 刷新 404 怎么修

让未知前端路由回到 index.html

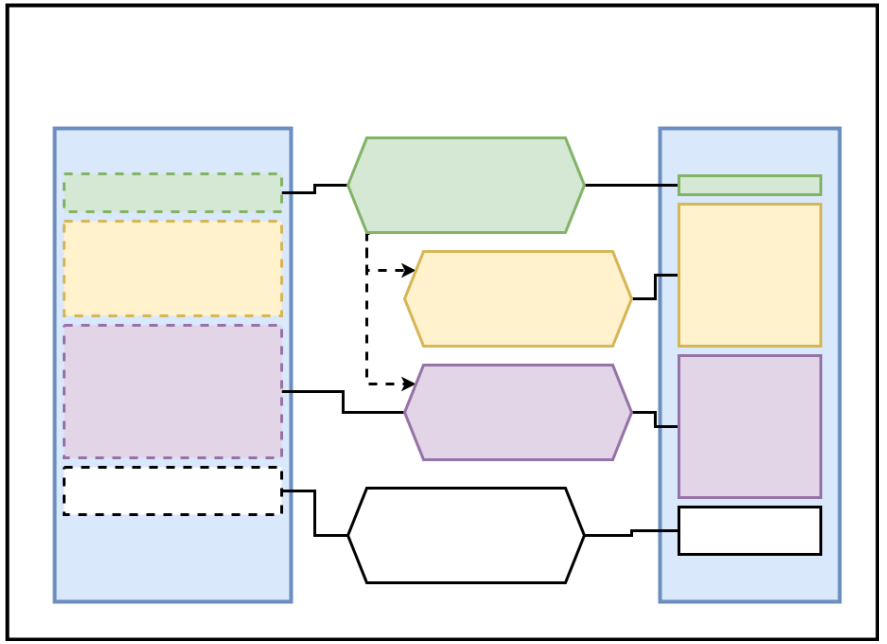
```
location / {
    try_files $uri $uri/ /index.html;
}
```

- React Router / Vue Router 的 History 模式由浏览器管理路由。
- 用户直接访问 /chat/123 时，Nginx 找不到同名文件会返回 404。
- try_files 让前端路由接管；/api 保持独立代理规则。

6.4 FastAPI 在宝塔中的运行模型

锁定依赖、仅本机监听、交给进程守护

关键点	生产环境含义
依赖	遵循 pyproject.toml 与 uv.lock
启动入口	使用项目真实的 ASGI 入口
监听地址	仅监听 127.0.0.1:8000
健康检查	提供轻量 /health
生命周期	开机启动、异常重启与日志



一个服务可能由多个独立进程共同承担请求。

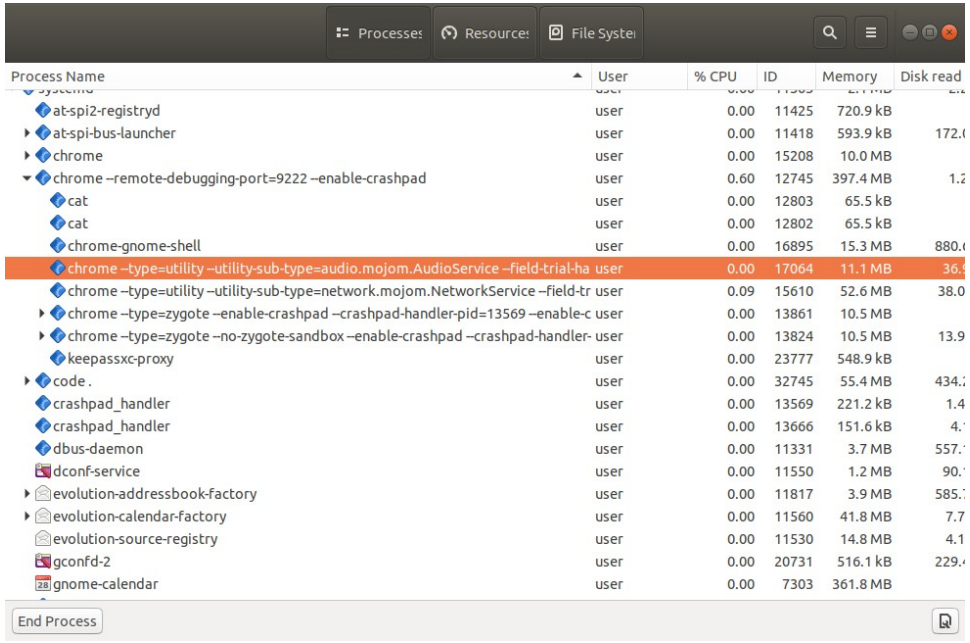
来源：FastAPI Docs · Deployment Concepts

两个阶段 先证明应用能启动，再交给守护进程长期运行

6.5 进程守护需要配置什么

SSH 断开后仍运行，异常后能够恢复

- 工作目录与真实启动入口。
- Python 虚拟环境或 Node 版本。
- 本机监听端口与运行用户。
- 环境变量和受控 Env 文件。
- 开机启动与异常退出重启。
- 标准输出、错误日志和保留周期。



守护工具关注进程、用户、资源和退出状态。

来源：FastAPI Docs · Server Processes

版本差异 不同宝塔版本入口可能显示为 Python 项目、应用管理器或进程守护

6.6 Nginx 把/api 转给 FastAPI

同域部署减少 CORS 与 Cookie 问题

```
location /api/ {
    proxy_pass http://127.0.0.1:8000/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

- 访问 `https://app.example.com/api/health`，Nginx 转给 FastAPI。
- 注意 `proxy_pass` 末尾斜杠会影响路径改写，必须现场验证真实路由。
- 后端启用 Proxy Headers 后，才能正确感知外部协议与客户端信息。

来源：NGINX Docs · Reverse Proxy

6.7 反向代理界面要看懂的字段

面板字段最终都会变成 Nginx 配置

- 代理目录决定哪些请求被匹配。
- 目标 URL 指向本机应用端口。
- Host 与转发头保留域名、协议和客户端信息。
- API 通常关闭缓存；静态资源可单独配置缓存。
- WebSocket / SSE 需要额外确认连接与超时设置。



来源：宝塔官方文档·反向代理

6.8 前后端联调的验证顺序

后端、Nginx、浏览器逐层检查

1. `curl http://127.0.0.1:8000/health`：后端进程是否正常。
2. `curl -i http://127.0.0.1/api/health`：Nginx 反代是否正常。
3. 浏览器打开域名：前端静态资源是否正常。
4. F12 Network 查看 `/api/...` 请求、状态码、响应体。
5. 后端日志确认同一请求是否到达、耗时与错误。

定位法 在哪一层第一次失败，问题就位于该层或它的下一跳

6.9 Node 后端与 FastAPI 的部署差异

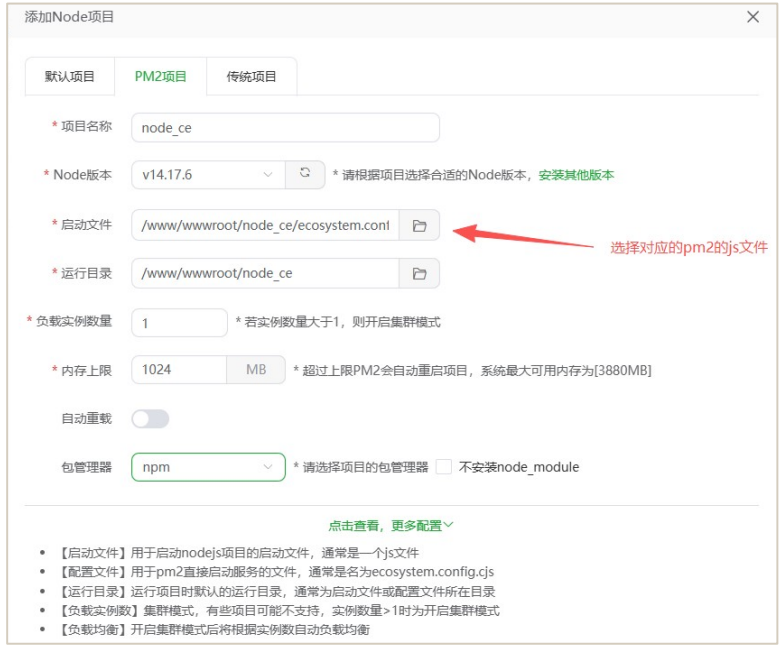
运行时不同，部署模型相同

对比项	Node 后端	FastAPI 后端
依赖锁定	`pnpm-lock.yaml`	`uv.lock`
构建阶段	NestJS / TypeScript 常需构建	通常直接运行 Python 源码
运行时	Node.js + pnpm / PM2	Python 虚拟环境 + ASGI Server
本机端口	常见 `127.0.0.1:3000`	常见 `127.0.0.1:8000`
进程管理	宝塔 Node 项目 / PM2	Python 项目 / 应用管理器
公网入口	都由 Nginx 反向代理，不直接暴露业务端口	

共同模型 锁定依赖 + 本机端口 + 进程守护 + Nginx 入口

6.10 Node 项目配置要认清的字段

项目路径、启动命令、端口和运行用户



添加 Node 项目·宝塔官方



配置启动端口·宝塔官方

来源：宝塔官方文档·Next.js 部署

pnpm 找不到命令时先确认 Node 版本与 PATH，避免重复安装

6.11 环境变量如何放到服务器

仓库保存变量名，服务器保存真实值

类型	处理方式
前端公开配置	构建时注入；默认视为用户可见，不放 Secret
后端普通配置	项目环境变量或权限受控的 Env 文件
数据库密码 / API Key	最小权限、定期轮换、日志脱敏
JWT Secret	使用足够随机值，变更会影响现有 Token
上传目录	独立持久目录，权限给运行用户并纳入备份

权限 Env 文件只允许部署用户 / 运行用户读取，避免通过静态站点暴露

07

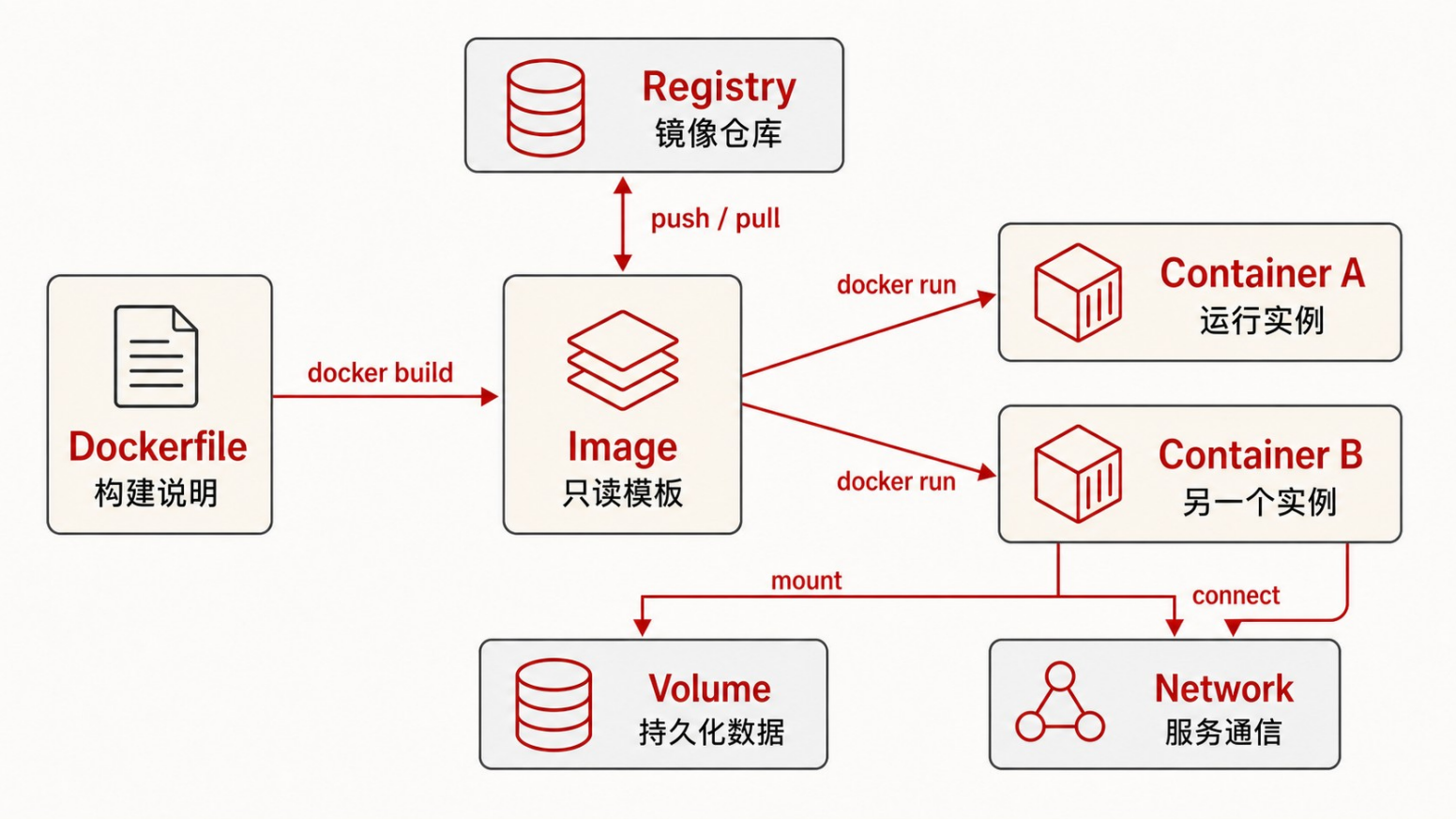
Docker 基础与部署模型

镜像、容器、网络、卷与 Compose

7.1	Docker 核心部件的关系	50
7.2	Claude Code：准备 Docker 部署	51
7.3	最小 Dockerfile 读法	52
7.4	Docker 层与缓存	53
7.5	Build、Run、Port 三个动作	54
7.6	容器排错的四个命令	55
7.7	Docker Network 让服务用名字互访	56
7.8	Docker Compose 管理多容器应用	57

7.1 Docker 核心部件的关系

Dockerfile、Image、Container、Registry、Volume 与 Network



同一 Image 可启动多个 Container；Volume 保存数据，Network 负责容器通信。

7.2 Claude Code：准备 Docker 部署

一段可直接复制的专业提示词

你是资深 DevOps 工程师。请先扫描当前仓库，再为生产服务器准备 Docker 部署。

已知技术栈：
- 前端：Node.js + pnpm；判断是静态构建还是 Next.js / Node SSR。
- 后端：Python FastAPI + uv，必须遵循 pyproject.toml 与 uv.lock。
- 数据库：MySQL 8.4；公网入口由 Nginx 提供，同域/api 转发后端。
请按顺序完成：
1. 检查目录、构建脚本、启动命令、端口、环境变量、迁移、健康检查和上传目录。
2. 先输出部署拓扑、假设、风险与将要新增/修改的文件，然后直接实现。
3. 为每个服务创建生产级 Dockerfile：多阶段构建、锁文件安装、最小运行镜像。
4. 创建.dockerignore，排除.git、.env、依赖、缓存、构建物和本地虚拟环境。
5. 创建 compose.yaml：frontend、api、db（镜像固定 mysql:8.4）、healthcheck、restart、内部 network；将 MySQL /var/lib/mysql 挂载到 named volume；只把网关绑定到宿主机 127.0.0.1。
6. 创建 Nginx 配置：SPA fallback 或 SSR 反代、/api、代理头、WebSocket / SSE。
7. 创建.env.example，只保留变量名和安全示例，禁止写入任何真实 Secret。
8. 提供数据库迁移、启动、日志、健康检查、更新、备份与回滚命令。
约束：
- 不猜测缺失的业务信息；会改变架构时先提出问题。
- 保留现有可用配置，修改前说明原因；不要提交.env、密码或密钥。
- 执行 docker compose config，并检查构建命令与服务间连接。
- 最后汇总文件路径、运行步骤、验证方法、风险和待确认项。

数据库选择 Docker 内 MySQL 适合小型项目；重要生产数据优先使用云数据库

7.3 最小 Dockerfile 读法

每条指令都在描述镜像构建过程

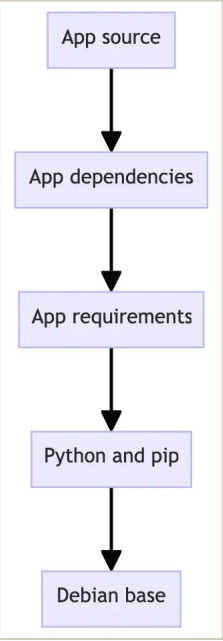
```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "app.main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

- RUN 在构建时执行；CMD 在容器启动时执行。
- 容器内服务监听 0.0.0.0，才能从容器网络访问。
- JSON Exec Form 能更正确地接收停止信号。

7.4 Docker 层与缓存

把变化少的步骤放前面

1. 复制依赖清单与锁文件。
2. 安装依赖，生成可缓存层。
3. 再复制频繁变化的源码。
4. 最终镜像只保留运行产物。



每条构建指令形成可复用的只读层。

来源：Docker Docs · Understanding Image Layers

常见错误 先 `COPY . .` 再安装依赖，任何源码变化都会让依赖层缓存失效

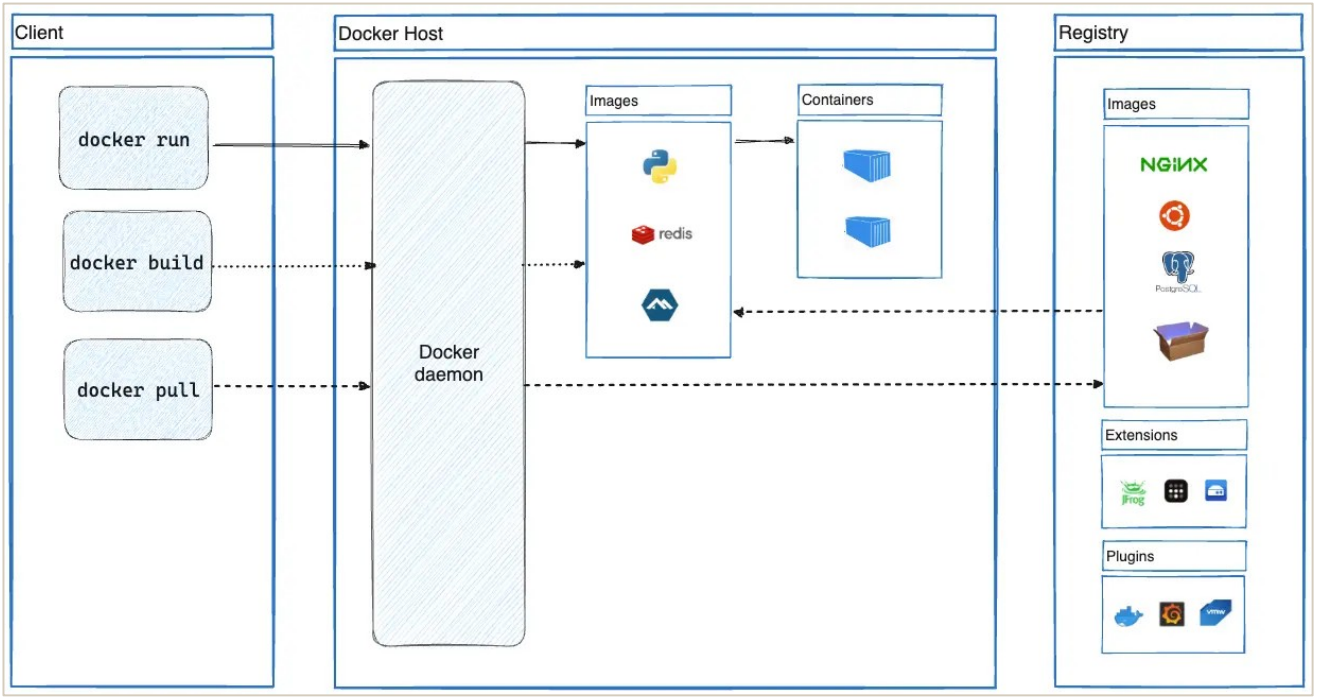
7.5 Build、Run、Port 三个动作

左边是宿主机端口，右边是容器端口

```
docker build -t customer-api:1.0 .

docker run -d \
  --name customer-api \
  -p 127.0.0.1:8000:8000 \
  --restart unless-stopped \
  customer-api:1.0
```

- 左侧是宿主机端口。
- 右侧是容器监听端口。
- 生产镜像使用明确 Tag。



来源：Docker Docs · Docker Architecture

7.6 容器排错的四个命令

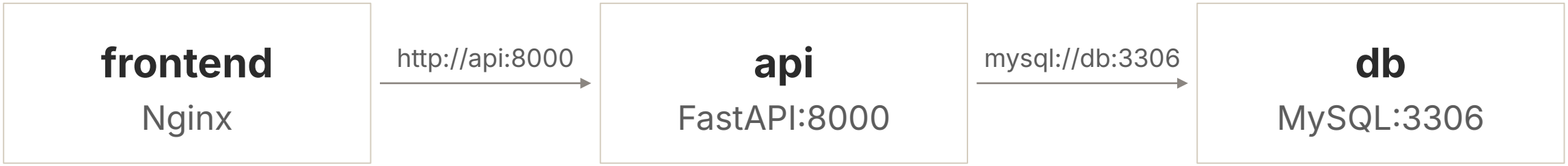
状态、日志、进程内检查、配置

```
docker ps -a
docker logs --tail 200 -f customer-api
docker exec -it customer-api sh
docker inspect customer-api
```

- `docker ps -a` 看退出状态和重启循环。
- `docker logs` 看应用标准输出与错误输出。
- `docker exec` 用于诊断，不应在容器内手工修改生产代码。

7.7 Docker Network 让服务用名字互访

Compose 会为项目创建默认网络



- 容器间不使用 `localhost`；`localhost` 只代表当前容器自己。
- 使用服务名 `api`、`db` 作为网络主机名。
- 只有需要给宿主机 / 公网访问的服务才映射 ports。

7.8 Docker Compose 管理多容器应用

一份 YAML 描述服务、网络、卷与配置

```
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

- `config` 先检查合并后的配置和变量替换。
- `down` 默认不删 Named Volume；加 `-v` 会删除数据卷，必须谨慎。
- Compose 适合单机部署，也是今天全栈 Docker 路线的主角。

来源：Docker Docs · Compose

08

域名、反向代理与 SSL 证书

域名找服务器，Nginx 分流，证书建立信任

8.1	A 记录与 CNAME 记录	59
8.2	用 dig 验证 DNS	60
8.3	一个域名如何承载前后端	61
8.4	代理头传递外部请求信息	62
8.5	WebSocket 与 SSE 代理配置	63
8.6	Let's Encrypt / ACME 自动签发	64
8.7	HTTP-01 与 DNS-01 怎么选	65
8.8	宝塔申请与部署 SSL	66
8.9	HTTP 跳转 HTTPS	67
8.10	Mixed Content 排错	68
8.11	CORS 配置	69
8.12	上线后验证域名与证书	70
8.13	证书续签与 HSTS	71

8.1 A 记录与 CNAME 记录

一个指向 IP，一个指向另一个主机名

类型	示例	适用
A	@ → 203.0.113.10	根域名指向 IPv4 服务器
AAAA	@ → IPv6	使用 IPv6 时
CNAME	www → example.com	子域名别名或第三方服务
TXT	验证字符串	域名验证、邮件安全、ACME DNS-01

- @ 通常代表根域名，www 代表 www.example.com。
- 修改记录后存在缓存与 TTL，不同递归 DNS 生效时间可能不同。

来源：Cloudflare DNS Docs · Record examples

8.2 用 dig 验证 DNS

同时检查 Host Header 与站点匹配

```
dig +short app.example.com
dig app.example.com A
nslookup app.example.com

curl -I http://203.0.113.10 \
  -H "Host: app.example.com"
```

- dig 验证域名解析结果。
- 带 Host Header 访问 IP，可区分 DNS 问题与 Nginx 站点配置问题。
- 使用 CDN / 代理 DNS 时，查询结果通常指向代理节点。

8.3 一个域名如何承载前后端

推荐同域 + 路径分流

方案	示例	特点
同域路径	`app.example.com/ + /api`	Cookie、CORS 最简单，课程默认
不同子域	`app.example.com + api.example.com`	边界清晰，需要配置 CORS / Cookie
业务端口	`example.com:8000`	不适合作为正式用户入口

统一入口：用户只访问 443，应用真实端口留在内部。

8.4 代理头传递外部请求信息

协议、域名与客户端 IP

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

- 后端生成重定向 URL、记录审计日志时需要外部请求信息。
- FastAPI 位于可信代理后时使用 `--proxy-headers`。
- 只信任可信反向代理写入的 X-Forwarded-For。

来源：FastAPI Docs · Behind a Proxy

8.5 WebSocket 与 SSE 代理配置

AI 流式输出的超时与缓冲

```
# WebSocket
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";

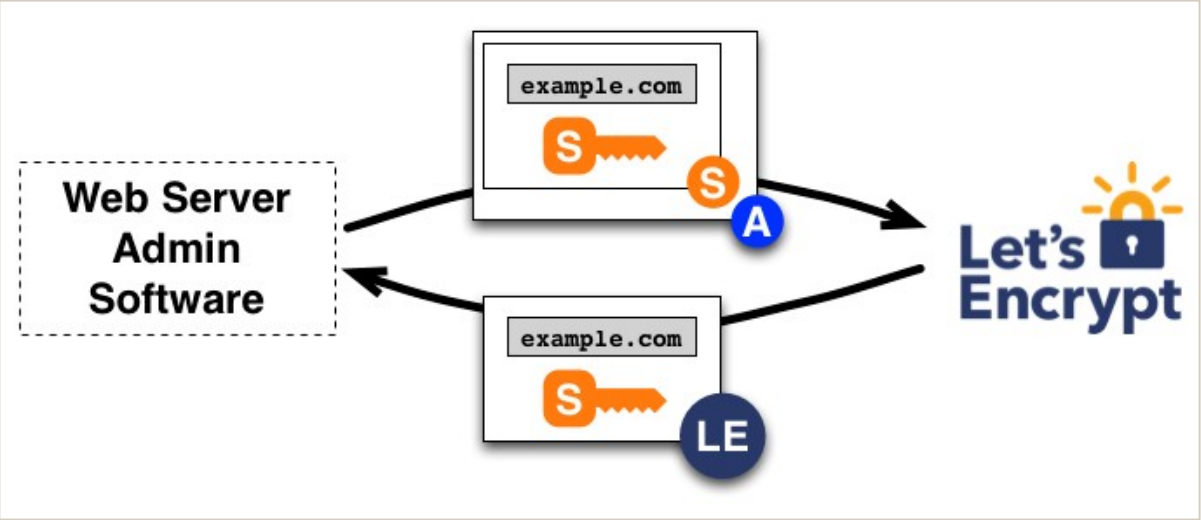
# Server-Sent Events / streaming
proxy_buffering off;
proxy_read_timeout 3600s;
```

- 普通 JSON 接口可以使用默认缓冲；流式响应需要尽快向客户端转发。
- 超时过短会让长回答中途断开；过长也要配合连接与资源限制。
- 在浏览器 Network 中确认数据持续到达。

8.6 Let's Encrypt / ACME 自动签发

验证域名控制权并定期续期

1. ACME Client 请求验证挑战。
2. 通过 HTTP 文件或 DNS 记录证明控制域名。
3. CA 从网络侧验证挑战。
4. 生成 CSR 并签发证书。
5. 客户端自动安装并定期续期。



来源：Let's Encrypt · How It Works

8.7 HTTP-01 与 DNS-01 怎么选

课堂单域名优先 HTTP-01

验证方式	要求	适用
HTTP-01	域名已指向服务器，80 端口可访问	单域名、普通网站，配置简单
DNS-01	能通过 DNS 添加 TXT 记录 / API	泛域名证书、80 不可达、集中证书管理

- HTTP-01 申请失败先查 DNS、80 端口、防火墙和站点匹配。
- DNS API Token 使用最小权限，并像生产密钥一样保护。

8.8 宝塔申请与部署 SSL

站点设置中完成申请、安装与续签



来源：宝塔官方文档 · 部署 SSL 证书

- 域名先正确解析到服务器。
- 进入站点 SSL 页面申请。
- 可选免费证书或上传已有证书。
- 开启强制 HTTPS 与到期提醒。

8.9 HTTP 跳转 HTTPS

80 端口保留验证与重定向

```
server {
    listen 80;
    server_name app.example.com;

    location /.well-known/acme-challenge/ {
        root /www/server/panel/vhost/certbot;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}
```

- 宝塔通常会自动生成对应配置，不必手写路径。
- 强制 HTTPS 后，前端 API、图片、字体和 WebSocket 都应使用安全协议。

8.10 Mixed Content 排错

HTTPS 页面中的 HTTP 请求会被拦截

典型错误 页面是 `https://app.example.com`，前端仍请求 `http://203.0.113.10:8000/api`。

- 浏览器会阻止不安全的 Fetch、XHR、Script、Stylesheet 等请求。
- 让 API、脚本、图片与 WebSocket 全部使用安全协议。
- 使用相对地址 `/api`，可同时避免协议、域名与环境差异。

来源：MDN · Mixed content

8.11 CORS 配置

浏览器对跨 Origin 请求的约束

场景	处理
同域 `/api`	同源请求，配置最简
`app.example.com` → `api.example.com`	后端明确允许前端 Origin
带 Cookie / Authorization	不能用任意 `*`；需精确 Origin 与 Credentials 配置
Postman / curl 能通，浏览器不通	优先检查 CORS 与 Preflight

安全边界 CORS 不能替代后端鉴权；攻击者可以不用浏览器直接请求 API

8.12 上线后验证域名与证书

检查跳转、证书、接口和安全头

```
curl -I http://app.example.com
curl -I https://app.example.com
curl -i https://app.example.com/api/health

openssl s_client \
  -connect app.example.com:443 \
  -servername app.example.com </dev/null 2>/dev/null |
openssl x509 -noout -subject -issuer -dates
```

- HTTP 应跳转 HTTPS；证书域名与有效期正确；API 返回预期 JSON。
- 浏览器控制台检查 Mixed Content、CORS、证书链与资源 404。

8.13 证书续签与 HSTS

自动续签、到期提醒与安全策略

- 免费证书通常周期较短，依赖自动续签；开启到期提醒。
- 续签失败常见原因：DNS 已切换、80 被关闭、验证路径被代理、DNS Token 失效。
- 确认 HTTPS 长期稳定后再考虑 HSTS。
- HSTS 配错会让浏览器拒绝回退 HTTP，证书错误时也无法绕过。

运维要求 持续监控自动续签与到期时间

来源：MDN · Strict-Transport-Security

09

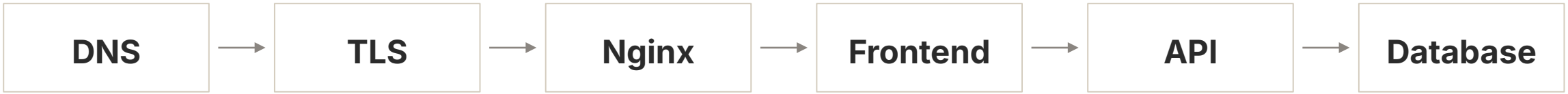
运维、排错、回滚与课堂实战

日志、排错、备份、更新与恢复

9.1	线上故障按层定位	73
9.2	日志分层看什么	74
9.3	502 Bad Gateway 决策树	75
9.4	404 有三种完全不同的来源	76
9.5	数据库连接失败怎么分层	77
9.6	服务器资源不足的症状	78
9.7	备份范围与恢复验证	79
9.8	单机部署的安全基线	80
9.9	一次规范发布包含哪些阶段	81
9.10	回滚触发条件	82
9.11	从手工部署走向 CI/CD	83
9.12	课堂综合实战	84
9.13	官方参考资料·协作、Docker 与本地模型	85
9.14	官方参考资料·宝塔、网络与 HTTPS	86
	交付完成	87

9.1 线上故障按层定位

从用户入口一路追到数据层



- 每层都用独立工具验证，找到“第一次失败”的位置。
- 页面报 500 时，从首次失败的层级开始处理。

9.2 日志分层看什么

同一请求最好能用 Request ID 串起来

日志层	关注字段	常见问题
Nginx Access	Host、Path、Status、耗时	请求是否到达、慢在哪
Nginx Error	upstream、connect、time-out	502、证书、配置
应用日志	Request ID、用户、异常栈	业务错误、第三方 API
Docker / 进程	Exit Code、Restart Count	启动失败、崩溃循环
MySQL	连接、慢查询、磁盘	密码、迁移、性能

日志纪律 不记录密码、Token、完整身份证件与模型密钥；生产错误要可行动

9.3 502 Bad Gateway 决策树

Nginx 到上游的连接失败

```
# 1. Is the process/container running?
docker compose ps
docker compose logs --tail 200 api

# 2. Is the port listening?
ss -lntp | grep 8000

# 3. Can the host/proxy reach it?
curl -i http://127.0.0.1:8000/health

# 4. Is Nginx config valid?
nginx -t
```

结论 本机 curl 都失败，先修应用；本机成功但域名失败，再查 Nginx 与防火墙

9.4 404 有三种完全不同的来源

先看响应内容和 Server Header

现象	可能来源	修复方向
刷新前端子路由 404	Nginx 找不到文件	SPA `try_files`
`/api` 返回前端 HTML	路由顺序 / 回退误吞 API	独立 `location /api/`
FastAPI JSON 404	路径被代理改写错误	检查尾斜杠与 root path
静态 JS / CSS 404	构建 base path 错	检查资源前缀与站点根目录

9.5 数据库连接失败怎么分层

网络、账号、Schema、迁移

1. 主机名是否正确：容器内用 db，宿主机进程可能用 127.0.0.1。
2. 端口是否正确，数据库是否真正 Ready。
3. 用户、密码、数据库名是否与初始化配置一致。
4. 用户是否有目标数据库权限，字符集是否符合项目要求。
5. 当前代码需要的迁移是否已执行。

容器陷阱 API 容器里的 `localhost:3306` 指向 API 容器自身

9.6 服务器资源不足的症状

构建时和运行时的瓶颈不同

```
free -h
df -h
du -sh /var/lib/docker/*
docker stats
top
```

- Node 前端构建常见内存峰值；MySQL、模型服务与多 Worker 会长期占内存。
- 磁盘可能被 Docker Image、Build Cache、日志和数据库共同吃满。
- OOM Kill 时应用像“突然消失”，需要查系统日志和容器退出码。

9.7 备份范围与恢复验证

数据库、上传文件、配置与证书

对象	备份方式	恢复验证
MySQL	定时逻辑备份 + 云快照	新库导入并跑关键查询
上传文件	Volume / 目录同步 / 对象存储版本	随机抽样文件可打开
Compose / Nginx	进入 Git	新服务器可复现
Env 变量名	`.env.example` + Secret 清单	新环境变量齐全
证书	可自动重新签发为主	续签任务与域名验证可用

3-2-1 重要数据至少 3 份、2 种介质、1 份异地；课堂项目也要养成习惯

9.8 单机部署的安全基线

默认拒绝，按需开放

- SSH 使用密钥，限制 root 登录
- 面板入口限制来源并启用 HTTPS
- 公网只开放 22 / 80 / 443
- MySQL、Redis、API 端口不暴露公网
- 运行进程使用非 root 用户
- 系统与依赖定期更新
- Secret 最小权限并定期轮换
- 数据库与上传文件定期备份
- 日志与磁盘空间设告警
- 删除离职成员与过期 Deploy Key

现实边界 一台服务器适合小型项目和教学；业务增长后再拆数据库、对象存储与负载均衡

9.9 一次规范发布包含哪些阶段

发布前、中、后都要有证据

阶段	动作
准备	分支已合并、测试通过、记录版本、备份、确认回滚点
构建	锁文件安装、生产构建、镜像打 Tag、扫描敏感信息
迁移	执行兼容性数据库迁移，失败则停止
发布	重启 / 替换进程或容器，保持变更范围最小
验证	Health、首页、登录、数据库、关键 API、日志
观察	错误率、响应时间、资源、用户反馈
收尾	记录结果，删除临时权限，更新变更日志

9.10 回滚触发条件

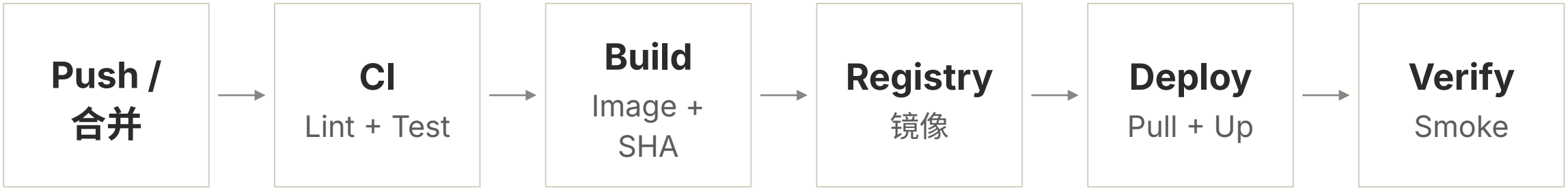
提前定义阈值与决策人

- 健康检查持续失败，服务无法启动。
- 登录、支付、核心对话等关键链路不可用。
- 错误率或响应时间明显超过基线。
- 数据写入异常、权限越权、Secret 泄漏等安全事件。
- 修复时间不可控，继续在线排查会扩大影响。

决策原则 先恢复服务，再分析根因；回滚本身也需要验证与记录

9.11 从手工部署走向 CI/CD

今天先把手工 SOP 做对，再自动化



- 自动化把已验证的 SOP 固化为可重复流程。
- CI 使用与本地一致的 pnpm / uv 版本与锁文件策略。

9.12 课堂综合实战

把 Day 3 / Day 4 项目真正交付给另一个人访问

1. 确认上线前修复已经合并到主分支。
2. 购买服务器后，完成宝塔基础环境配置。
3. 选择宝塔手工路线或 Docker Compose 路线部署。
4. 配置域名、`/api` 反向代理与 SSL。
5. 另一组使用手机网络访问并完成关键流程测试。
6. 主动制造一次错误端口，按日志定位并恢复。
7. 更新一个版本，再回滚到上一个版本。

9.13 官方参考资料·协作、Docker 与本地模型

- Pro Git Book · Git 基础与分支
- GitHub Docs · GitHub Flow
- Docker Docs · Compose
- pnpm Docs · Working with Docker
- uv Docs · Using uv in Docker
- Unsloth Docs · Qwen3.6 本地推理教程
- Unsloth Docs · Unsloth Studio
- llama.cpp · 本地推理与 OpenAI 兼容服务

9.14 官方参考资料·宝塔、网络与 HTTPS

- 宝塔面板官方文档
- 宝塔 · 反向代理
- 宝塔 · 部署 SSL 证书
- 宝塔 · Node.js PM2 部署
- NGINX Docs · Reverse Proxy
- Cloudflare Docs · DNS
- Let’s Encrypt · How It Works
- MDN · Mixed Content
- 工业和信息化部 · 互联网信息服务备案相关规定

交付完成

稳定交付，持续运行

A

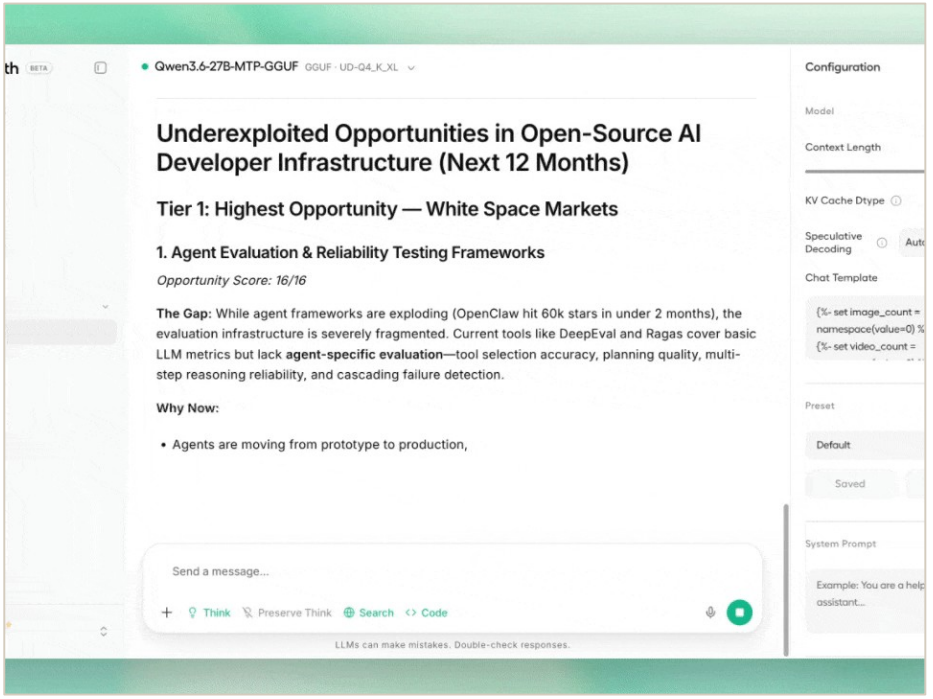
拓展·Qwen3.6 本地模型部署

	拓展·Qwen3.6 本地模型部署	89
A.1	先选 Studio，再决定是否下沉到 llama.cpp	90
A.2	安装并启动 Unsloth Studio	91
A.3	搜索、选量化、下载	92
A.4	运行前先对齐推理模式与参数	93
A.5	Studio 不只是聊天框	94
A.6	用 llama.cpp 进入可脚本化路线	95
A.7	用 llama-server 提供 OpenAI 兼容接口	96
A.8	MTP 是可选加速，不是固定开关	97

拓展·Qwen3.6 本地模型部署

基于 Unsloth 官方推理教程，结束页之后按需选讲

27B	4-bit 约需 18 GB 总内存
35B-A3B	4-bit 约需 23 GB 总内存



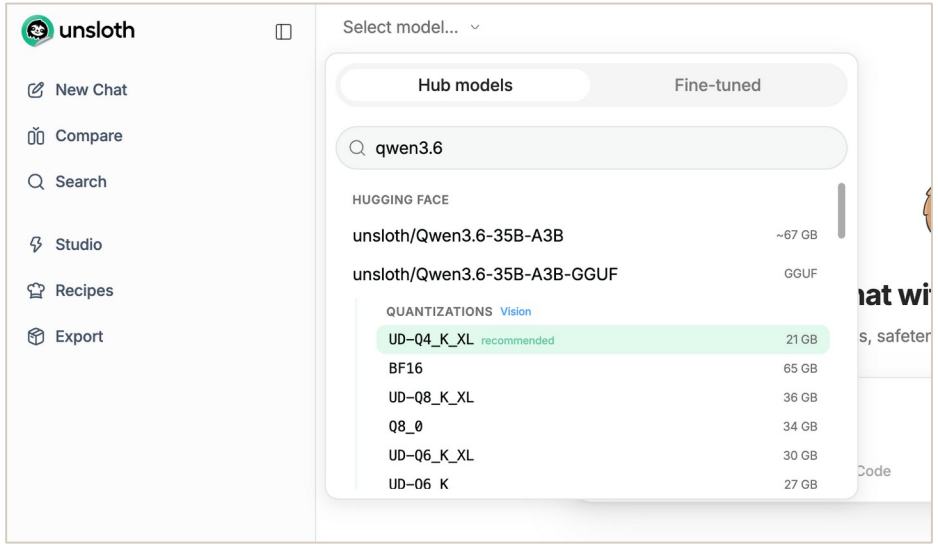
来源：Unsloth Docs · Qwen3.6 Inference Tutorials

课堂路线 Studio 跑通后，再下沉到命令行服务

A.1 先选 Studio，再决定是否下沉到 llama.cpp

可视化路线适合教学，命令行路线适合集成

模型·4-bit	总内存
Qwen3.6-27B	约 18 GB
35B-A3B	约 23 GB



Studio 直接列出量化版本与预计内存

来源：Unsloth Docs · Usage Guide

上下文 最大 256K；课堂先从 16K 起步

资源 RAM + VRAM 应高于量化文件大小

A.2 安装并启动 Unsloth Studio

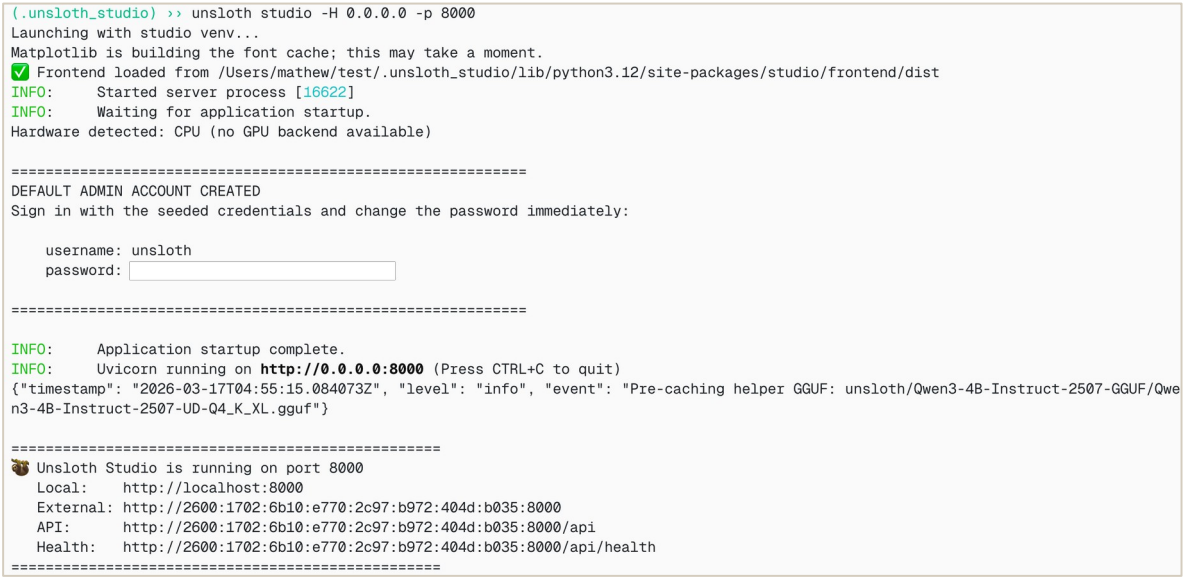
浏览器打开 127.0.0.1:8888，先完成最短闭环

```
# macOS / Linux / WSL
curl -fsSL https://unsloth.ai/install.sh | sh

# 只监听本机
unsloth studio -H 127.0.0.1 -p 8888

# 临时安全远程访问
unsloth studio --secure
```

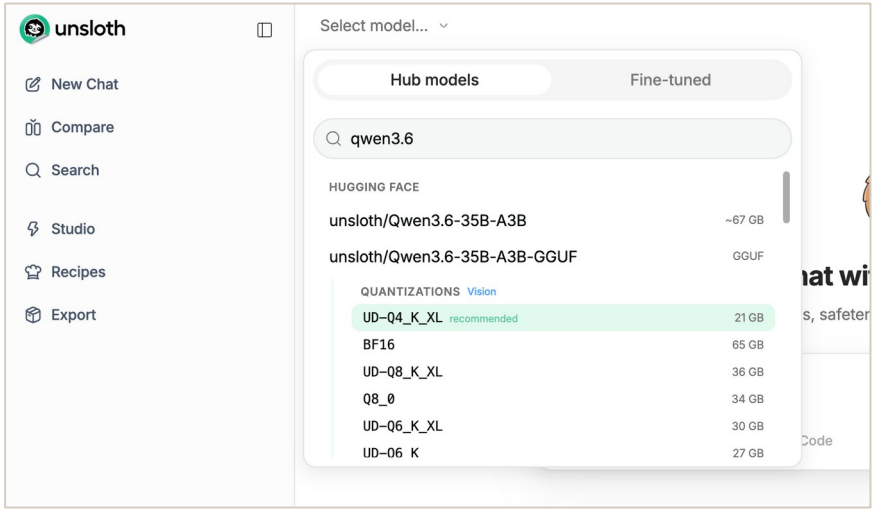
- Windows PowerShell 使用官方 `install.ps1`。
- 首次启动先设置 Studio 登录密码。



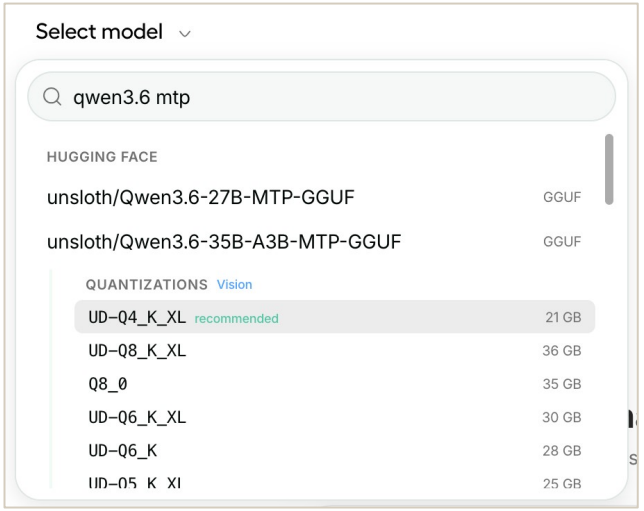
来源：Unsloth Docs · Install and Launch Unsloth Studio

A.3 搜索、选量化、下载

课堂默认 UD-Q4_K_XL；需要加速时再尝试 MTP



搜索 Qwen3.6 并选择 Dynamic 4-bit



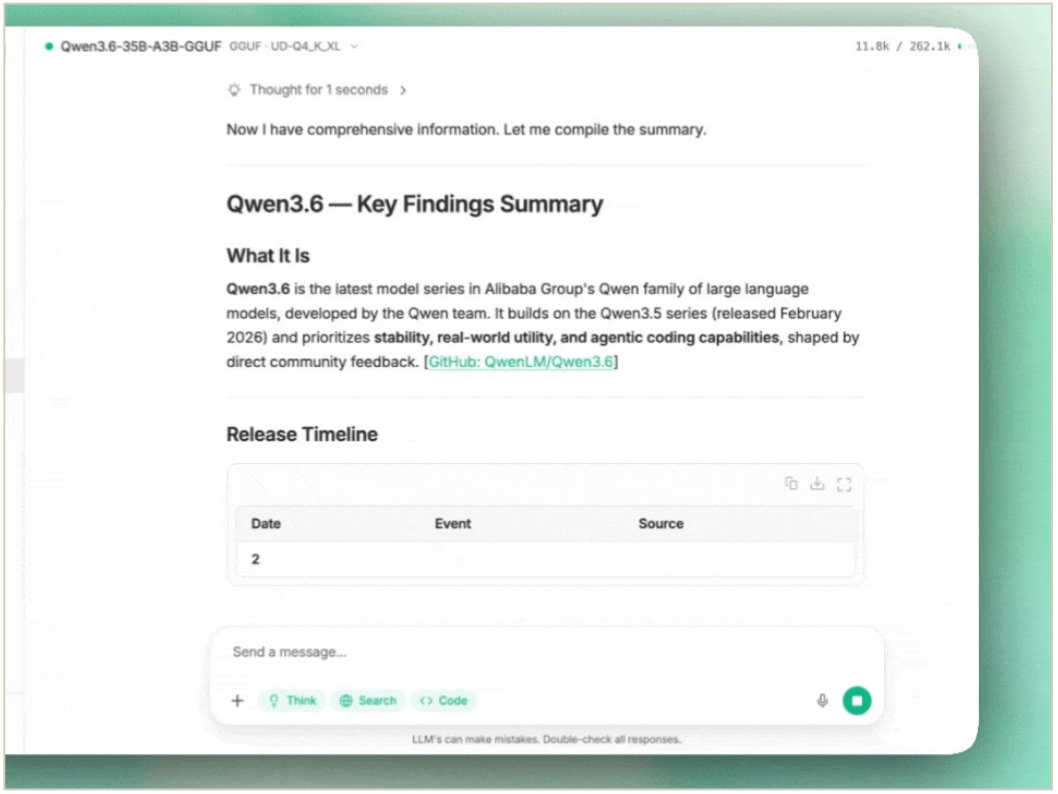
MTP 版本会单独列出

- 普通课堂优先 27B + UD-Q4_K_XL，资源更可控。
- MTP 通常多预留约 1 GB RAM / VRAM，并实测草稿 Token 数。

来源：Unsloth Docs · Search and Download Qwen3.6

A.4 运行前先对齐推理模式与参数

Thinking、精确编码、非思考模式使用不同温度



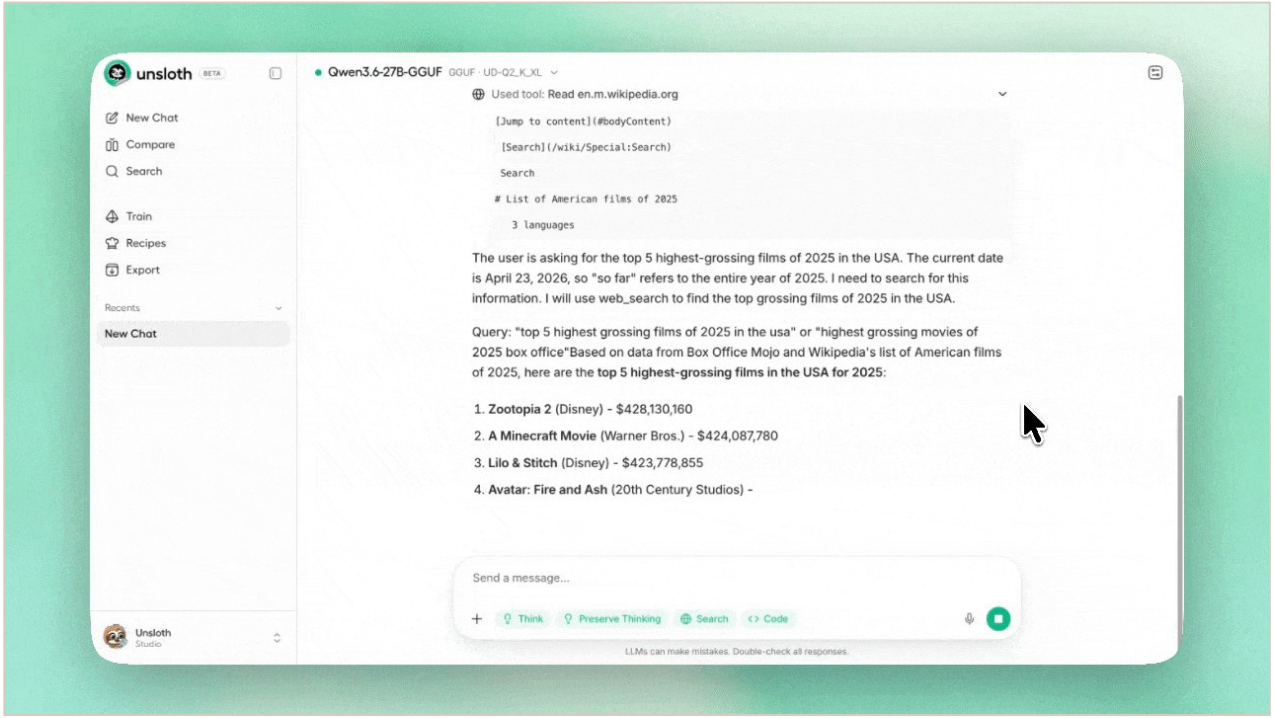
来源：Unsloth Docs · Recommended Settings

模式	Temp	Top-p
Thinking · 通用	1.0	0.95
Thinking · 编码	0.6	0.95
Non-thinking	0.7	0.8

- 三种模式均使用 `top-k=20`。
- 非思考模式设置 `presence penalty=1.5`。

A.5 Studio 不只是聊天框

同一界面验证搜索、工具调用、代码执行与持续对话



来源：Unsloth Docs · Run Qwen3.6 in Unsloth Studio

- 验证工具调用是否成功，而不只看最终答案。
- 记录首 Token、生成速度与内存占用。
- 连续推理再开启 Preserve Thinking。

A.6 用 llama.cpp 进入可脚本化路线

最新版 llama.cpp 可直接从 Hugging Face 拉取 GGUF

```
git clone https://github.com/ggml-org/llama.cpp
cmake llama.cpp -B llama.cpp/build \
  -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON
cmake --build llama.cpp/build --config Release -j \
  --target llama-cli llama-server

./llama.cpp/build/bin/llama-cli \
  -hf unsloth/Qwen3.6-27B-GGUF:UD-Q4_K_XL \
  --temp 1.0 --top-p 0.95 --top-k 20 --min-p 0.0
```

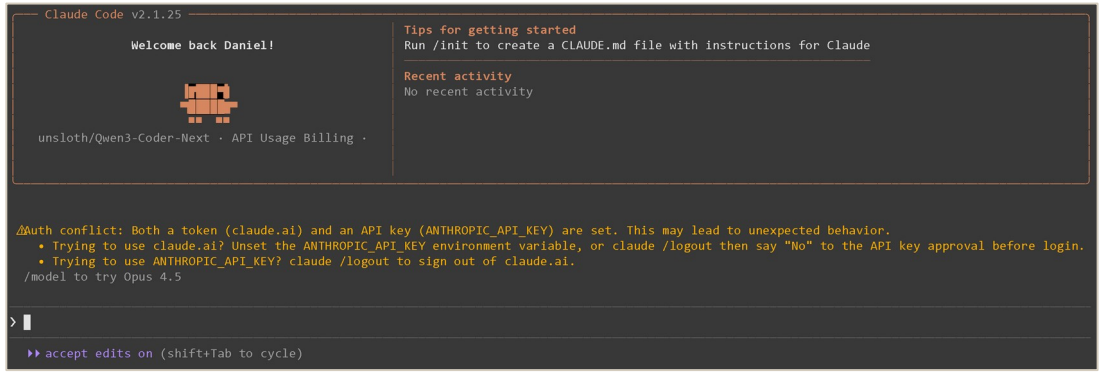
- 无 NVIDIA GPU 或 Apple Metal 设备使用 `-DGGML_CUDA=OFF`。
- 精确编码任务把温度改为 0.6；缓存目录可用 `LLAMA_CACHE` 固定。
- 避免 CUDA 13.2；官方页面提示使用低于 13.2 或 13.3。

来源：Unsloth Docs · Llama.cpp Guide

A.7 用 llama-server 提供 OpenAI 兼容接口

业务代码只替换 Base URL 与模型名

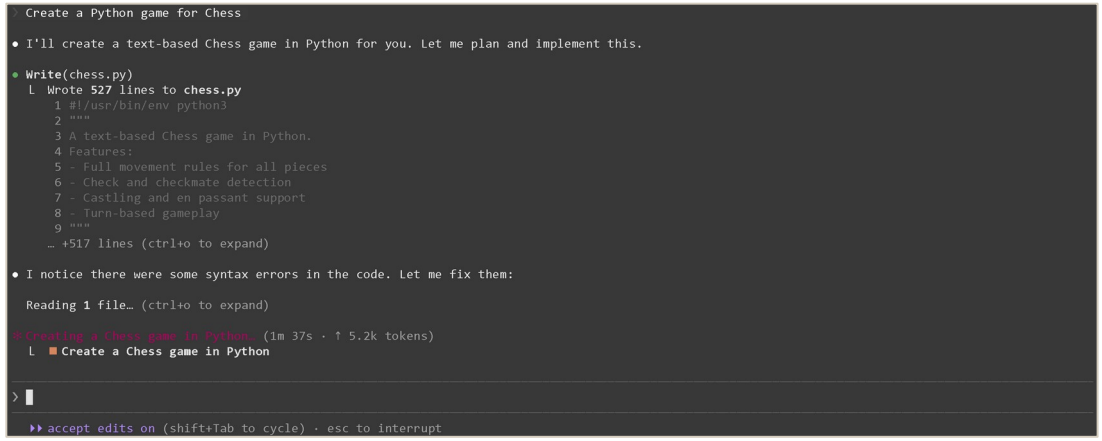
```
./llama.cpp/build/bin/llama-server \
  -hf unsloth/Qwen3.6-27B-GGUF:UD-Q4_K_XL \
  --alias qwen3.6-local \
  --ctx-size 16384 --port 8001 \
  --chat-template-kwargs \
  '{"enable_thinking":false}'
```



Python SDK：

```
http://127.0.0.1:8001/v1
```

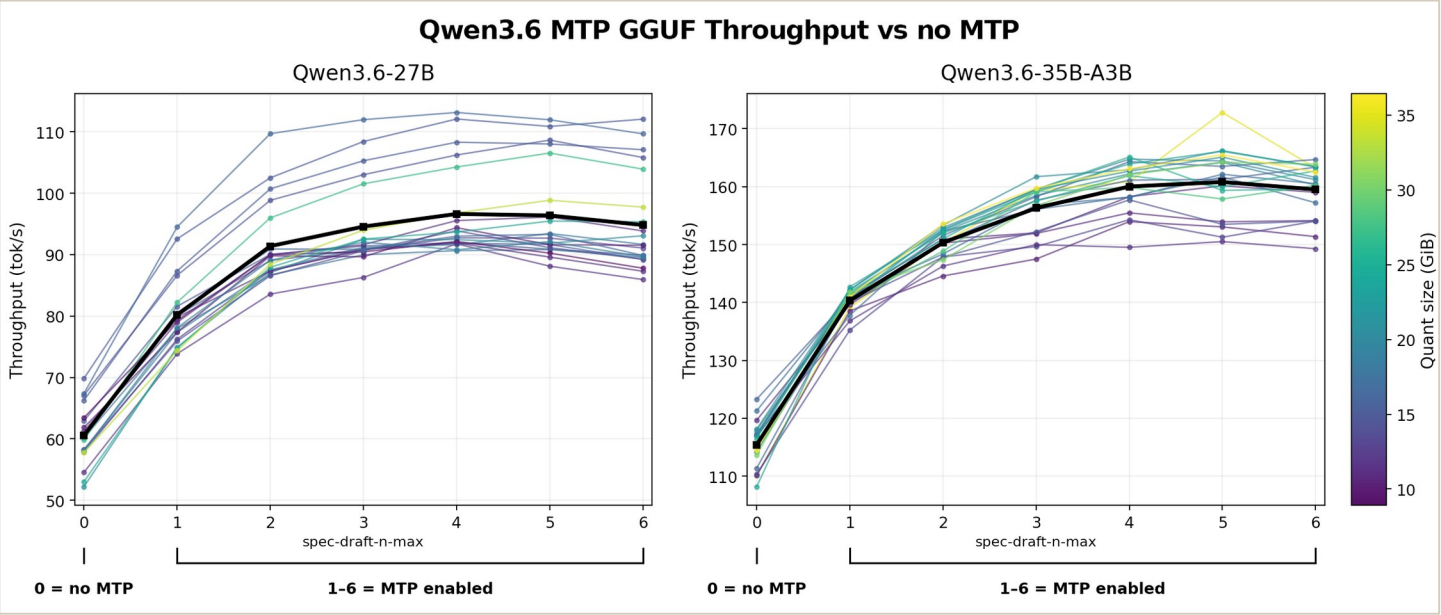
默认仅本机访问；远程调用必须增加鉴权与 HTTPS。



来源：Unsloth Docs · Llama-server and OpenAI Completion Library

A.8 MTP 是可选加速，不是固定开关

草稿 Token 数依硬件实测，常见起点是 2



来源：Unsloth Docs · Qwen3.6 MTP Guide and Benchmarks

- 教程报告约 1.4–2.2 倍生成加速。
- 草稿 Token 数从 1–6 逐个测试。
- 常见起点是 2，最终看实测速度与内存。